



Wrocław
University
of Science
and Technology

Hybrid Neighborhood Evaluation in Tabu Search and Simulated Annealing for the Permutation Flow Shop Problem

Remigiusz Wojewódzki¹ Wojciech Bożejko¹

¹ Department of Control and Quantum Computing, Faculty of Information and Communication Technology, Wrocław University of Science and Technology, Poland

remigiusz.wojewodzki@gmail.com wojciech.bozejko@pwr.edu.pl

Introduction

- The **Permutation Flow Shop Problem (PFSP)** schedules n jobs on m machines in the same order; the goal is to minimise the makespan $C_{\max}$. It is NP-hard for $m \geq 3$.
- For metaheuristics, the choice of the **neighborhood structure** — the set of moves explored each iteration — matters as much as the framework itself.
- We compare three neighborhoods of increasing granularity inside **Tabu Search (TS)** and **Simulated Annealing (SA)**: the Adjacent swap, the composite *Fibonacci* neighborhood, and the recursive Dynasearch.
- All under identical wall-clock budgets on the Taillard benchmark, measuring the relative percentage deviation (PRD) of $C_{\max}$ to the best-known value.

Problem description

The PFSP models a **production line**: n jobs pass through m machines in the *same fixed order*.



Given a processing-time matrix $P = [p_{k,j}]_{m \times n}$ with **integer** entries $p_{k,j} \in \mathbb{Z}_{\geq 0}$, a permutation π fixes the job order. The completion times obey

$$C_{k,j} = \max\{C_{k,j-1}, C_{k-1,j}\} + p_{k,\pi(j)},$$

for $k = 1, \dots, m, j = 1, \dots, n$, with the objective

$$\min_{\pi} C_{\max}(\pi) = C_{m,n}(\pi).$$

Three neighborhood structures

- Adjacent swap (baseline)**. Swap positions $(i, i+1)$, $i = 1, \dots, n-1$. Cheap to evaluate — many iterations, but only shallow local moves.
- Composite Disjoint-Adjacent (*Fibonacci*)**. A dynamic program selects a set of *non-overlapping* adjacent swaps minimising the joint $\sum_i \Delta_i$ and applies them at once — a bounded multi-step descent in a single iteration.
- Dynasearch (recursive)**. Considers all segments $[i, j]$, swapping their endpoints $\pi(i) \leftrightarrow \pi(j)$; a DP picks non-overlapping improving segments. $O(n^2)$ candidates — deep but costly.

Related very-large-scale neighborhoods (six-operator / global-local) trade breadth for per-iteration cost in the same spirit.

Per-iteration cost

Neighborhood	Candidates	Total per iter.
Adjacent	$n - 1$ swaps	$O(mn)$
Fibonacci	$n - 1$ swaps + DP	$O(mn^2)$
Dynasearch (rec.)	$O(n^2)$ segments	$O(mn^3)$

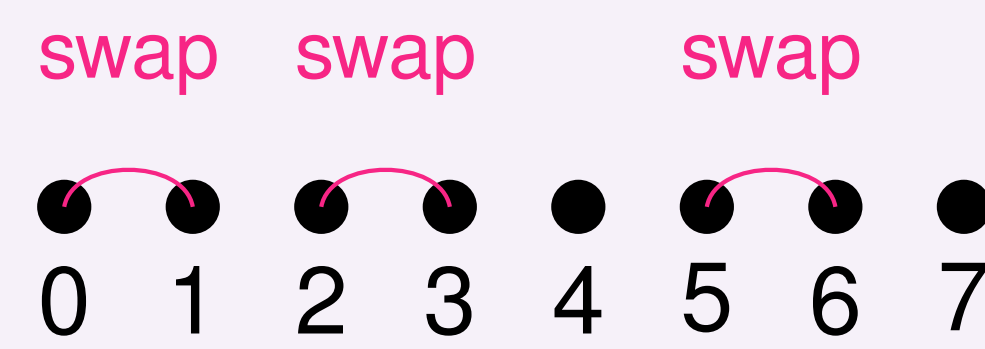
Fibonacci explores an **exponentially-sized** move set (F_{n+1} valid subsets) at **quadratic** cost — the sweet spot between cheap Adjacent and expensive Dynasearch.

The Fibonacci move

A move is a subset $S \subseteq \{1, \dots, n-1\}$ of adjacent-swap positions with the **non-overlap** constraint

$$i \in S \implies i+1 \notin S,$$

and all selected swaps $\pi_i \leftrightarrow \pi_{i+1}$ are applied **simultaneously**.



The number of valid subsets in a permutation of length n equals the Fibonacci number F_{n+1} — hence the nickname. The optimal subset is found by a single linear-time DP pass over the precomputed swap deltas:

$$\text{dp}[i] = \min(\text{dp}[i+1], \Delta_i + \text{dp}[i+2]).$$

Metaheuristic frameworks

Tabu Search (TS). Tabu list keyed by primitive moves or composite signatures; tenure $T_{\text{Tabu}} = 10$; an aspiration criterion accepts a tabu move when it improves the global best.

Simulated Annealing (SA). Multiplicative cooling $T \leftarrow \alpha T$ with floor and **time-based reheating**: after τ_{stagn} ms of no improvement, $T \leftarrow \min(T_0, rT)$. Standard acceptance:

$$\text{Pr}(\text{accept}) = \begin{cases} 1 & \text{if } \Delta < 0, \\ \exp(-\Delta/T) & \text{otherwise.} \end{cases}$$

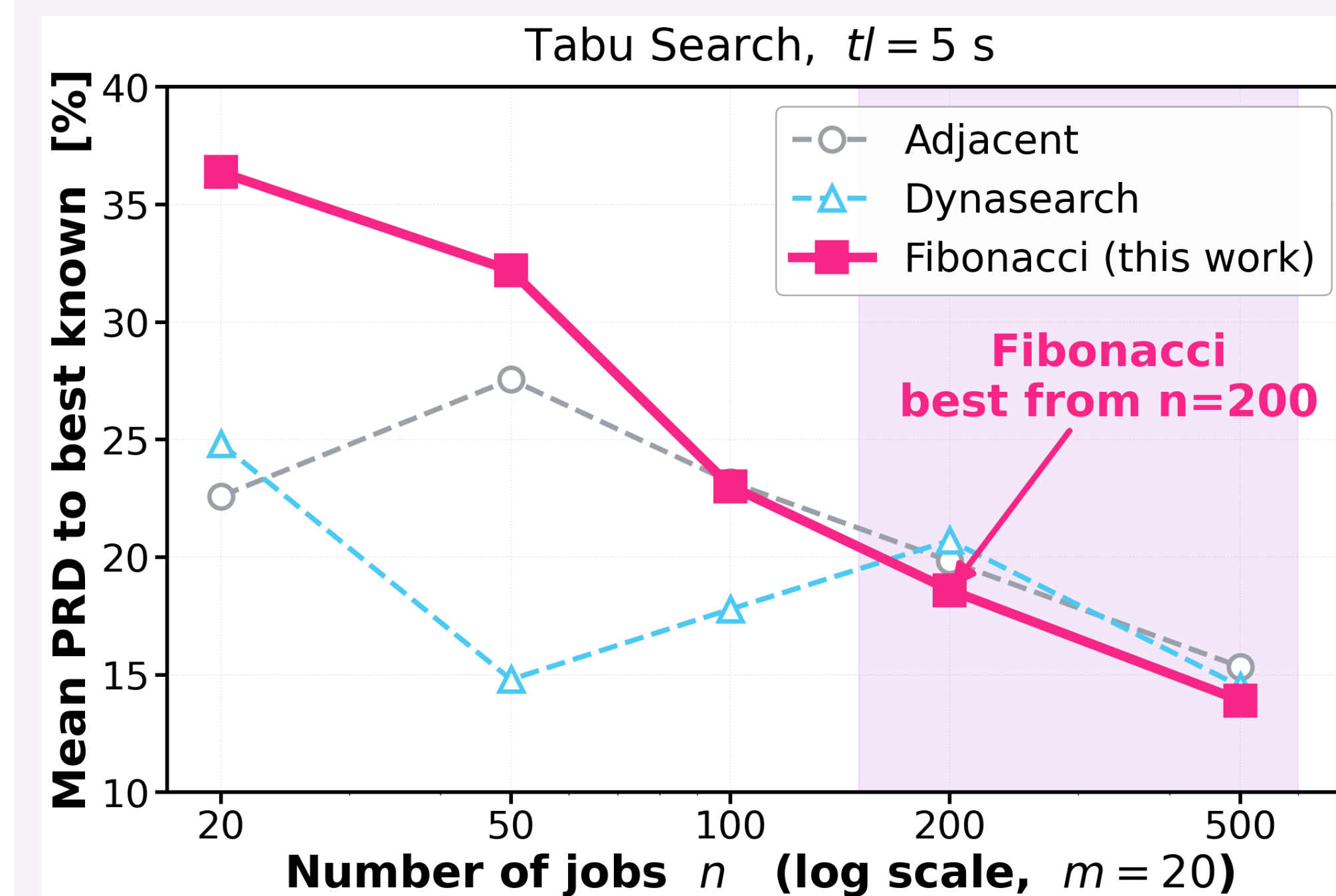
Both frameworks plug in the same neighborhoods, so any performance gap is attributable to the neighborhood, not the metaheuristic.

Key result — $n = 200, m = 20, tl = 5$ s, Tabu Search

Adjacent	13.19 %
Fibonacci	12.71 % ← best
Dynasearch	13.20 %

Fibonacci improves over Adjacent by **0.48 pp** and over Dynasearch by **0.49 pp** — at merely quadratic per-iteration cost. The crossover happens at $n = 200$, where Dynasearch's $O(mn^3)$ cost no longer fits the budget.

Where Fibonacci wins — PRD vs. n



Small n : Adjacent / Dynasearch dominate. **From $n = 200$** : the compound *Fibonacci* move pulls ahead and stays best at $n = 500$.

Selected references

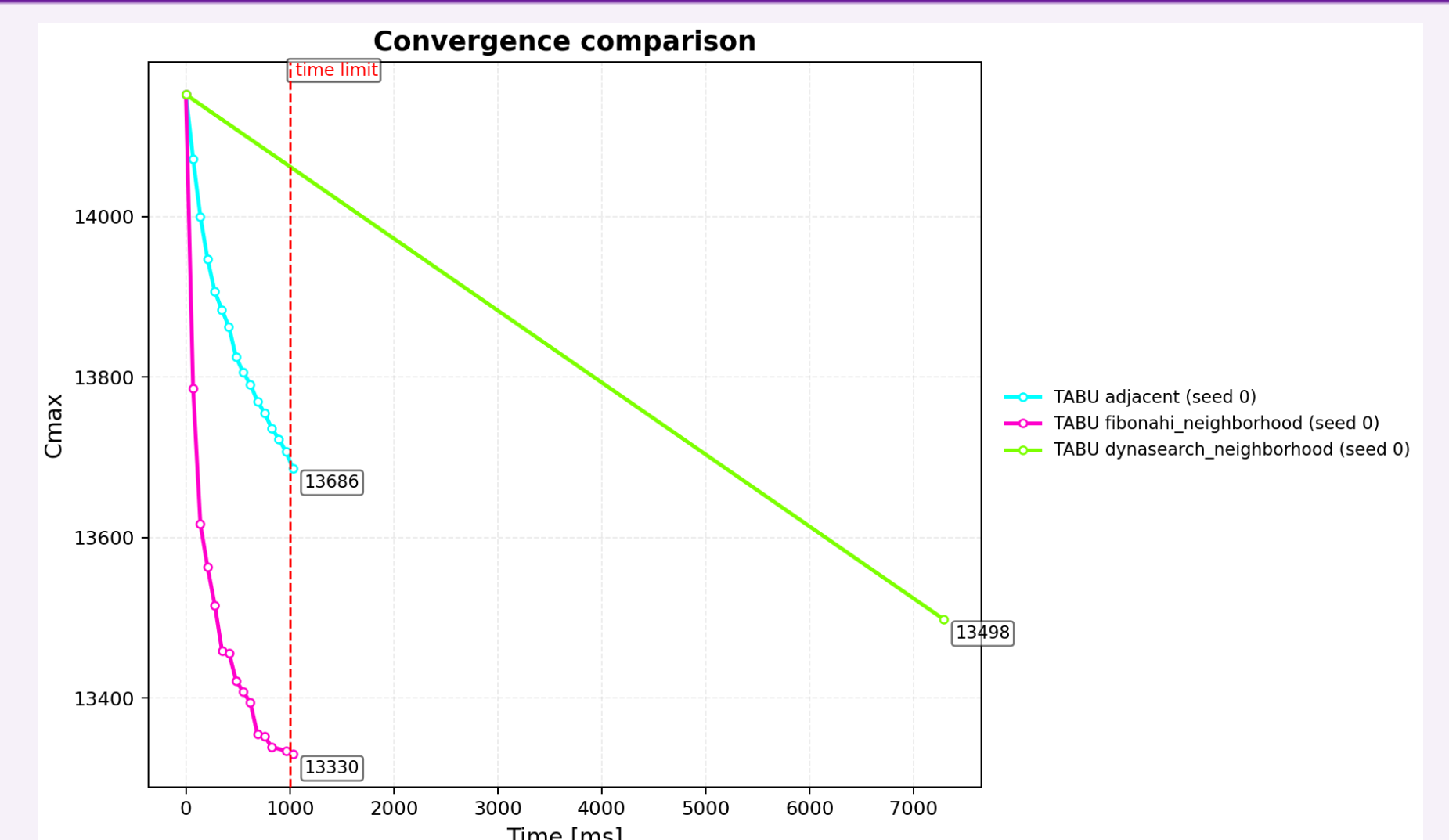
- [1] Taillard, E. *Benchmarks for basic scheduling problems*. Eur. J. Oper. Res. (1993).
- [2] Johnson, S. M. *Optimal two- and three-stage production schedules*. Nav. Res. Logist. Q. (1954).
- [3] Nowicki, E., Smutnicki, C. *A fast tabu search algorithm for the permutation flow-shop problem*. Eur. J. Oper. Res. (1996).
- [4] Glover, F. *Future paths for integer programming and links to artificial intelligence*. Comput. Oper. Res. (1986).
- [5] Kirkpatrick, S., Gelatt, C. D., Vecchi, M. P. *Optimization by simulated annealing*. Science (1983).
- [6] Congram, R., Potts, C., Van de Velde, S. *An iterated dynasearch algorithm...* (2002).
- [7] Bożejko, W., Wodecki, M. *On the theoretical properties of swap multimoves*. Oper. Res. Lett. (2007).
- [8] Ahuja, R., Orlin, J., Tiwari, A. *Very large-scale neighborhood search*. Oper. Res. (2002).

Per-instance PRD ($tl = 5$ s)

instance	n	m	fib _{TS}	fib _{SA}	dyn _{TS}	adj _{TS}
tai20_20	20	20	36.37	36.37	24.83	22.59
tai50_20	50	20	32.19	32.19	14.80	27.56
tai100_20	100	20	23.02	23.02	17.79	23.20
tai200_10	200	10	12.71	12.71	13.20	13.19
tai200_20	200	20	18.60	18.60	20.71	19.83
tai500_20	500	20	13.91	13.90	14.51	15.34

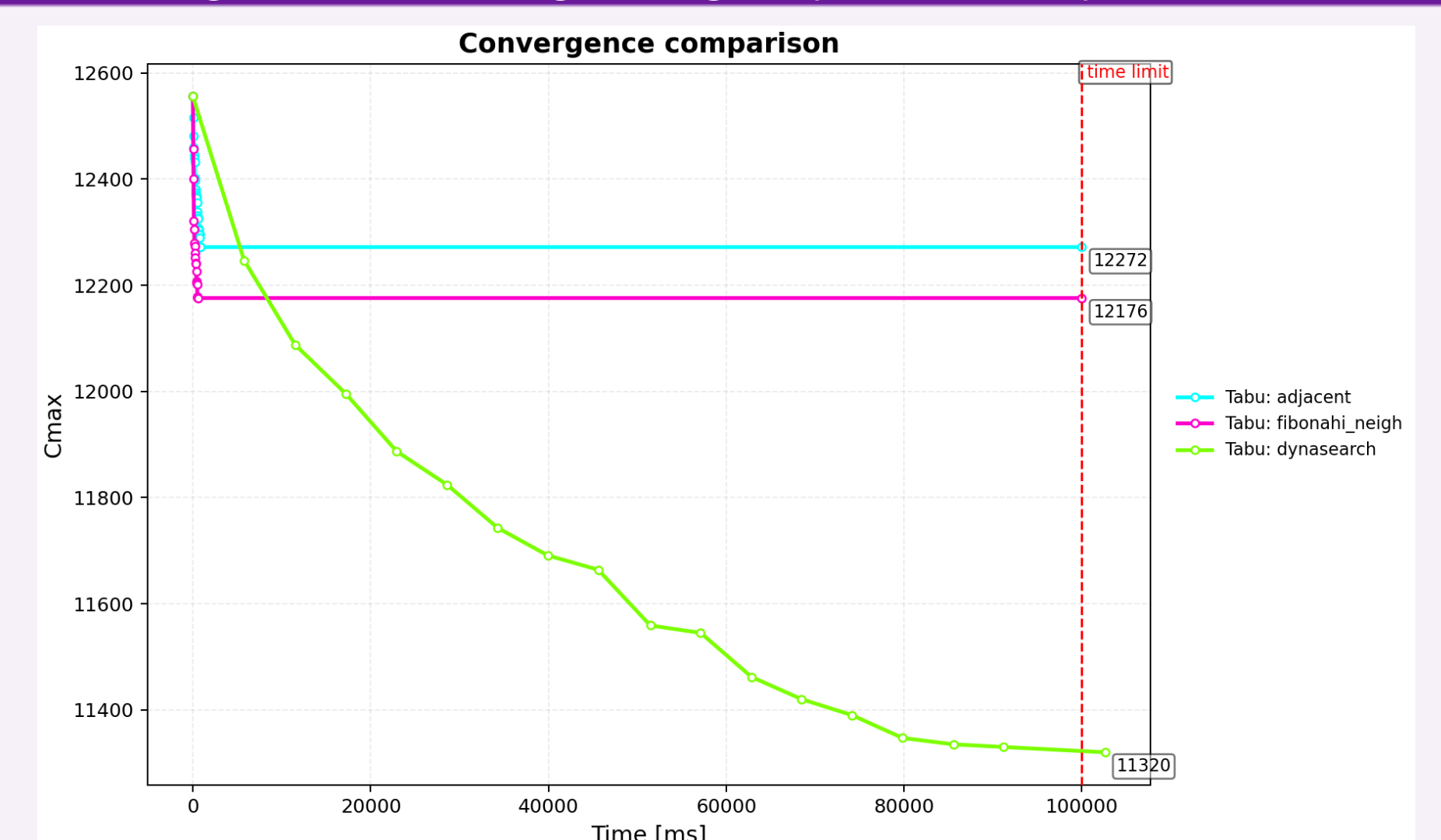
Best per row in bold. Fibonacci takes the lead from $n = 200$ onward.

Convergence — short budget ($tl = 1$ s)



tai200_20. Fibonacci (magenta) drops $C_{\max}$ in the first ~ 100 ms; Dynasearch (green) barely completes a single iteration within the budget.

Convergence — long budget ($tl = 100$ s)



tai200_10. Given two orders of magnitude more time, Dynasearch (green) keeps improving and eventually overtakes both — while Fibonacci and Adjacent plateau early.

Conclusions

- Composite *Fibonacci* accelerates **early** exploitation: a steeper initial $C_{\max}$ drop than Adjacent, by bundling non-overlapping improving swaps.
- It **dominates from $n = 200$** under short-to-medium budgets, where Dynasearch cannot finish even one iteration ($\sim 200\times$ the budget at $n = 500$).
- Once all $\Delta_i \geq 0$ the composite degenerates to a single swap and may plateau; **SA reheating** reliably restores diversification.
- Dynasearch remains best at small n or with large budgets.

Future work

- Incremental delta evaluation** of the composite move, cutting the per-iteration cost from $O(mn^2)$ toward $O(mn)$.
- Adaptive switching** between neighborhoods at runtime (e.g. Fibonacci early, Dynasearch once the budget allows).
- Probabilistic variant sampling** of the swap subset for additional diversification.
- Integration into stronger solvers (iterated local search, variable neighborhood descent) and comparison with state-of-the-art FSP methods.
- Quantum-assisted** Fibonacci selection on D-Wave Advantage (windowed QUBO, n up to 500) — companion paper at SOCO 2026.